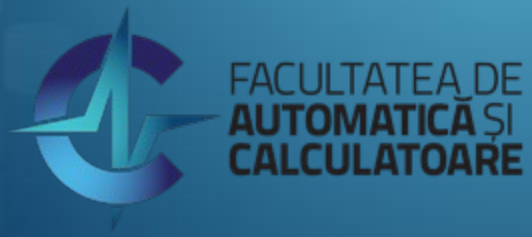


Algoritmi Paraleli și Distribuiți
Stabilirea Topologiei

Radu-Ioan Ciobanu
radu.ciobanu@upb.ro





Difuzarea mesajelor folosind sondaje

```
typedef tip_arb int[1:N,1:N];
chan sondaj[1:N](tip_arb,tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to N]{
   tip_arb arb, T;
   tip_mesaj m;
   bool top[1:N,1:N];

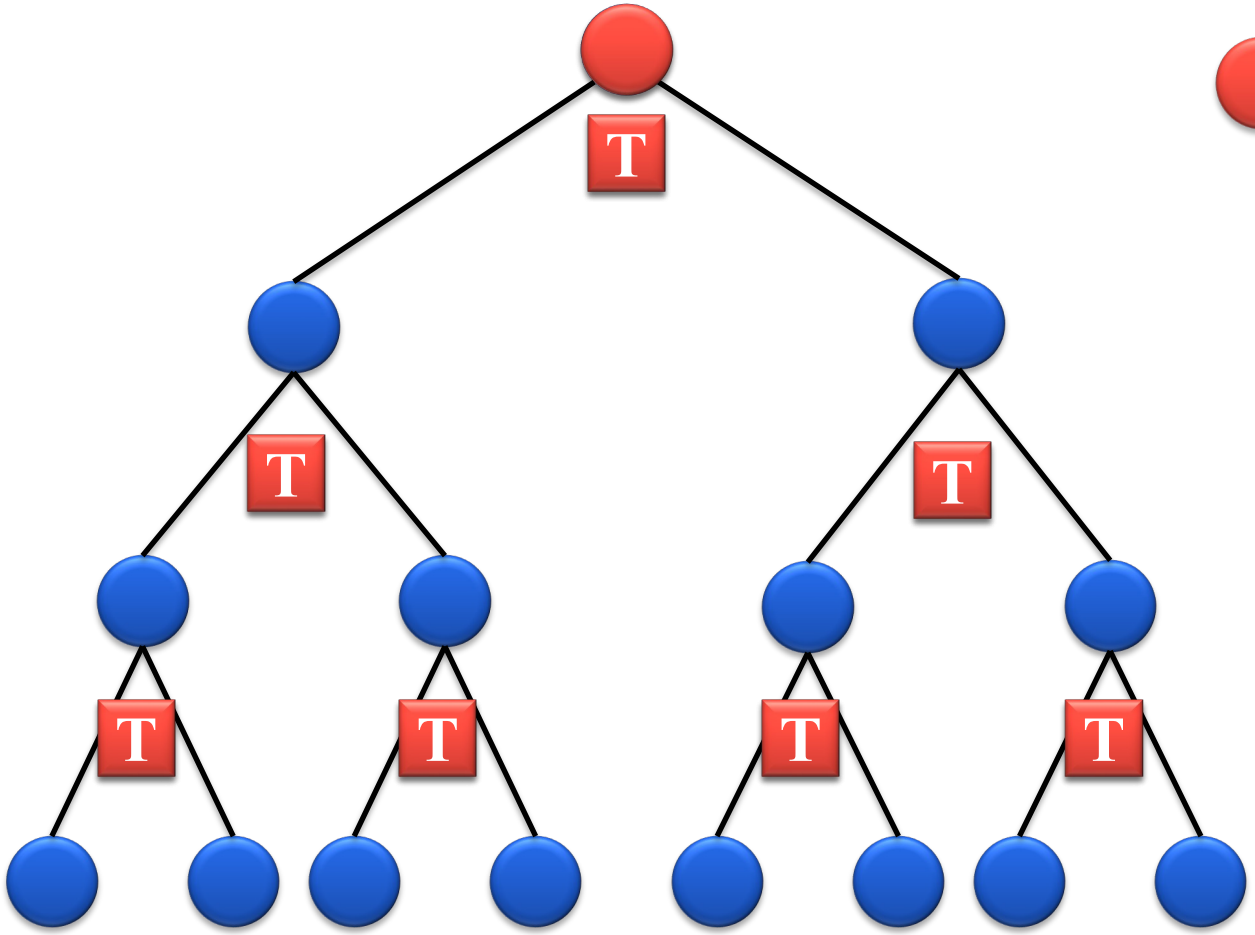
   if (inițiator == p) {
      calculeaza T pe baza top;
      m = mesaj_de_transmis
   } else
      receive sondaj[p](arb,m)

   for [q = 1 to N st q este fiul lui p in arb]
      send sondaj[q](arb,m);
}
```

top este o matrice cu topologia arborelui

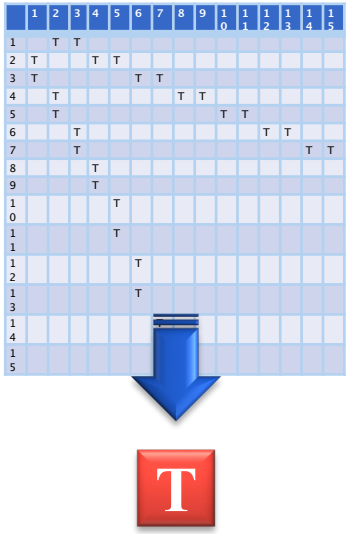
Numărul de mesaje = $N - 1$

Difuzarea mesajelor folosind sondaje



Inițiator

Matrice de adiacență



Difuzarea mesajelor prin inundare

```
chan sondaj[1:n](tip_mesaj);
const int inițiator = indexul nodului inițiator;

process Nod[p=1 to n]{
   bool leg[1:n] = vecinii_lui_p;
   int num = numărul_vecinilor, răspunsuri = num;
   tip_mesaj m;

   if (p <> inițiator) {
      receive sondaj[p](m);
      răspunsuri = răspunsuri - 1;
   else
      m = mesaj_de_transmis;

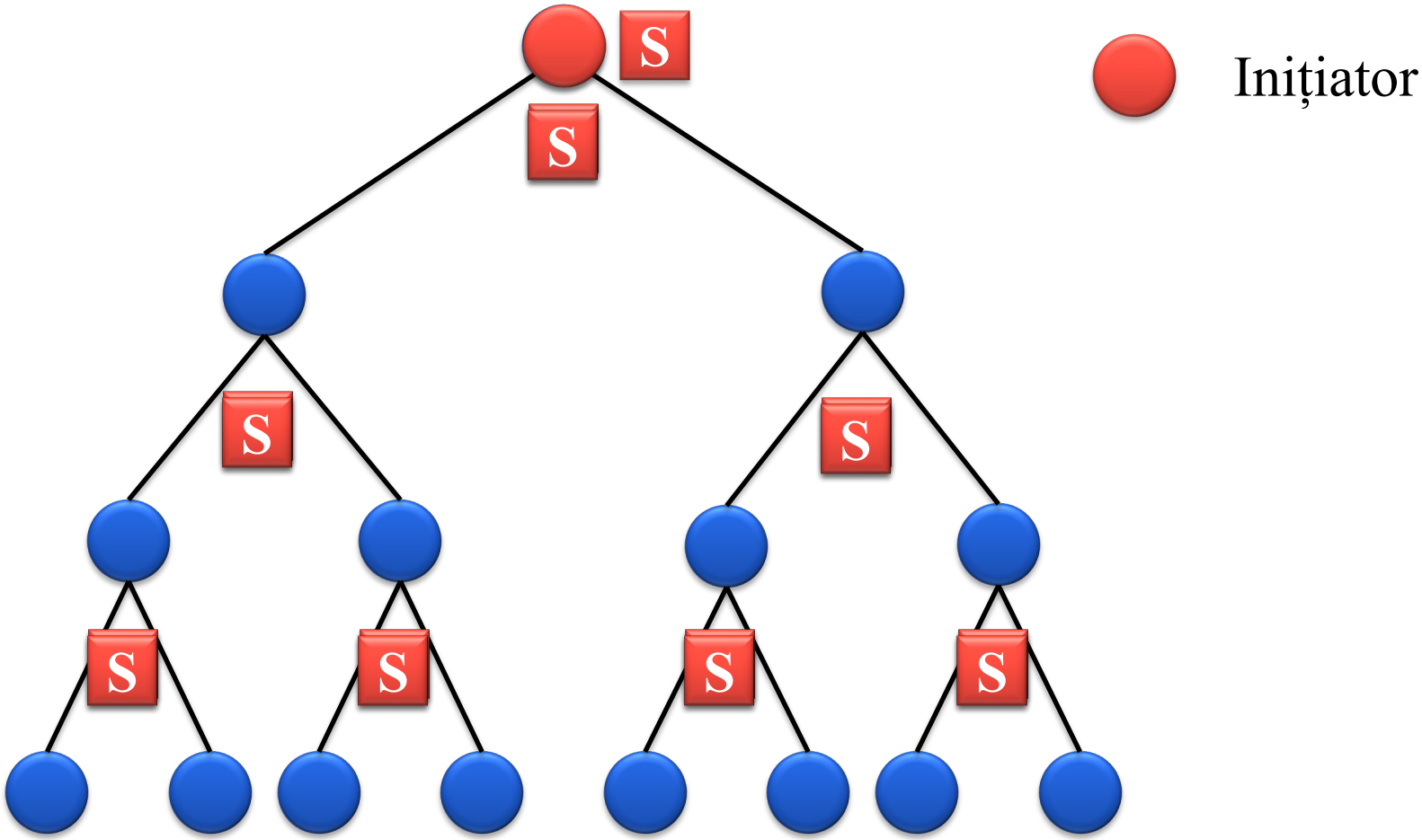
   for [q = 1 to n st leg[q]]
      send sondaj[q](m)

   for [q = 1 to răspunsuri]
      receive sondaj[p](m)
}
```

Numărul de mesaje = $2m$

(m = numărul legăturilor)

Difuzarea mesajelor prin inundare



Stabilirea topologiei unei reţele de procese

Definiţii

- Colecție de procese **Nod(p:1..N)**
- Vecinii nodului p **leg[1:N]:**
  - $leg[q] = TRUE$ dacă q este vecin cu p,
  - altfel $leg[q] = FALSE$.
- Relație simetrică:
  - pentru p, $leg[q] = TRUE \leftrightarrow$ pentru q, $leg[p] = TRUE$

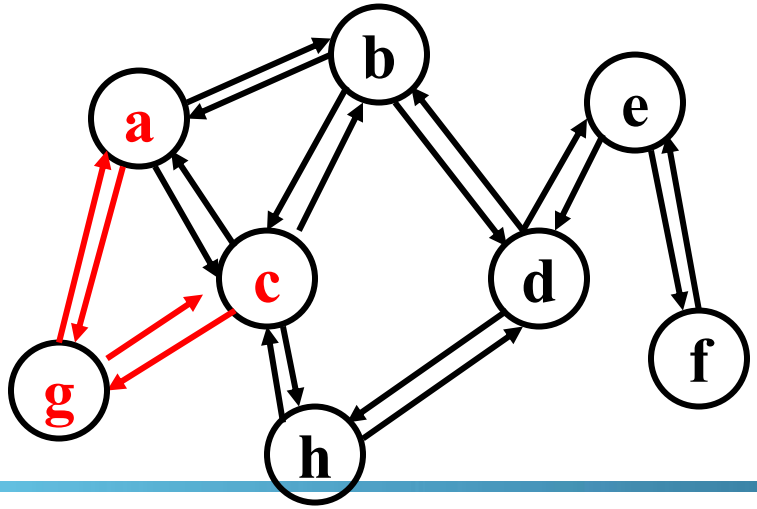
Problema

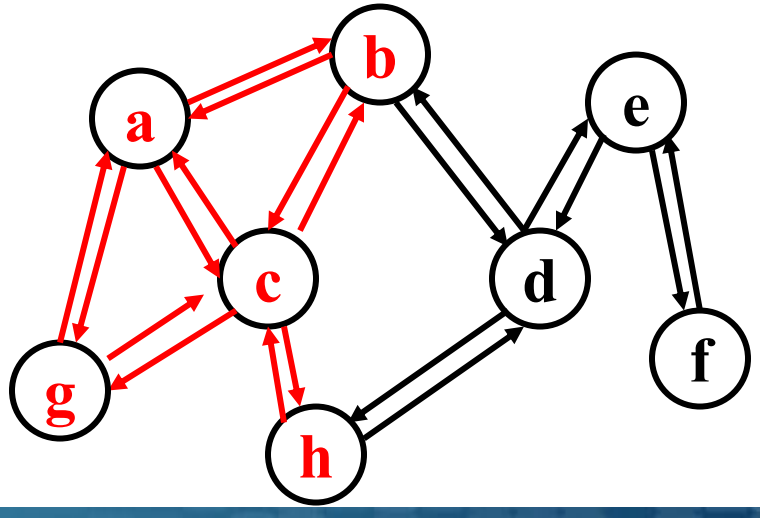
- Topologia reprezentată prin matricea de adiacenţe:
  - $top[i,j] = TRUE$, dacă i este vecin cu j
  - $top[i,j] = FALSE$, în caz contrar.
- Calculată pe baza cunoştinţelor locale
  - pentru orice p,q: $1 \leq p,q \leq N : top[p,q] = leg_p[q]$

Algoritmul pulsatiilor

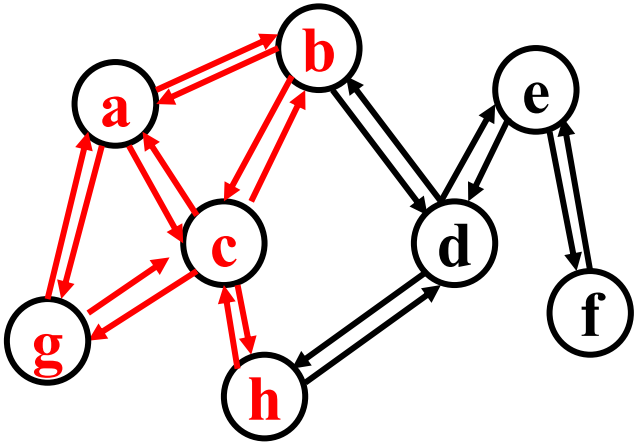
Algoritmul pulsaţiilor

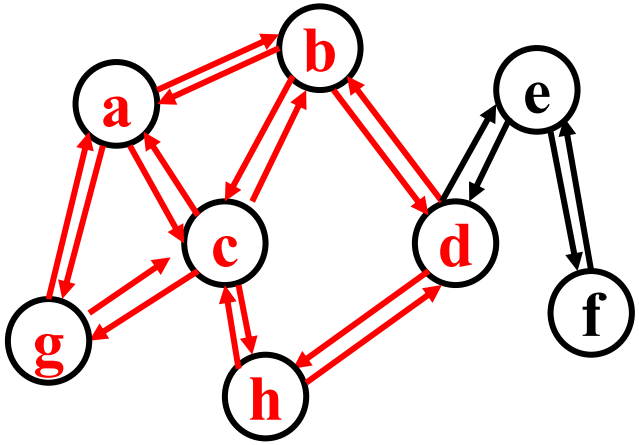
- Fiecare proces calculează singur topologia, folosind informaţiile provenite de la vecini
- Fiecare nod transmite vecinilor matricea sa de adiacenţă, top şi primeşte matricea de adiacenţă a fiecărui vecin
  - după o rundă completă, fiecare nod va dispune de informaţii de la nodurile vecine (figura stg. pentru nodul g)
  - După două runde complete, orice nod va avea informaţii de la noduri aflate la distanţa 2 (figura dr.), etc.
  - Dupa r runde sunt completate liniile din top corespunzătoare nodurilor q aflate la o distanţă <= r

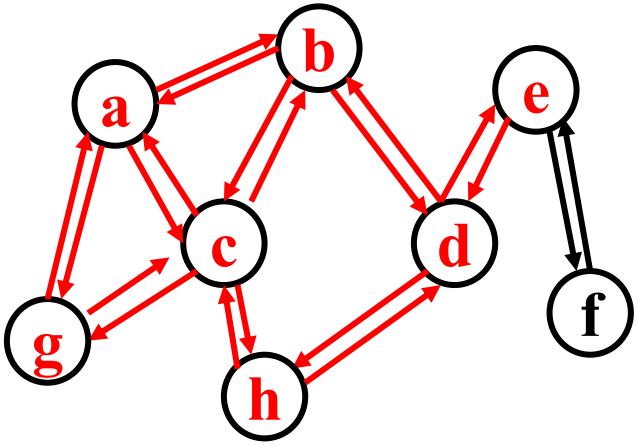


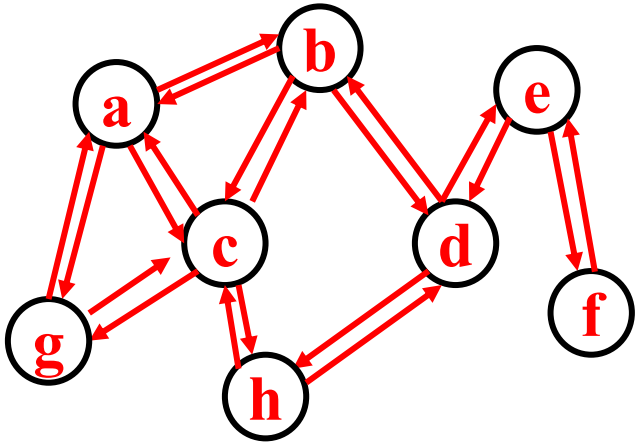


Algoritmul pulsaţiilor









Următorul predicat este satisfăcut pentru fiecare proces p:

RUNDĂ: oricare ar fi q cu 1<=q<=N: dist(p,q) <=r → top[q,*] este completat

dist(p,q) este distanţa de la p la q, adică lungimea căii celei mai scurte între cele două noduri.

Algoritmul pulsaţiilor

```
typedef tip_top bool[1:N,1:N];
typedef tip_leg bool[1:N];
chan topologie[1:N](tip_top);

process Nod[p=1 to n]{ tip_leg leg = vecinii_lui_p;
  int r = 0;
  tip_top top = ([N*N]FALSE), top_nou;
  top[p,1:N] = leg;
  /* actualizează liniile vecinilor */
  while (r < D) {
    /* transmite topologia curentă tuturor vecinilor */
    for [q = 1 to N st leg[q]]
      send topologie[q](top)
    for [q = 1 to n st leg[q]]{
      receive topologie[p](top_nou);
      top = top or top_nou;
    }
    r = r + 1;
  }
}
```

D = diametrul rețelei

Algoritmul pulsaţiilor (2)

- Neajunsuri:
  - În general, diametrul rețelei nu este cunoscut
  - Traficul inutil
- Rețea conexă
  - un nod cunoaşte întreaga topologie dacă fiecare linie ***top*** are cel puţin un element TRUE ⇒ mai execută o rundă și poate termina
  - evitare blocare ⇒ un proces comunică doar cu vecinii care nu au terminat

Algoritmul pulsaţiilor (3)

```
chan topo[1:N](tip_top);
process Nod[p=1 to N] {
    tip_leg leg = vecinii_lui_p;
    tip_leg activ = leg;                    /* vecinii activi */
    tip_top top = ([N * N]FALSE);
    bool gata = FALSE;
    int transm;
    bool qgata;
    tip_top top_nou;

    /* actualizează vecinii lui p */
    top[p, 1:N] = leg;
```

Algoritmul pulsaţiilor (4)

```
 while (NOT gata) {
       /* transmite topologia curentă tuturor
vecinilor */
       for [q = 1 to N st leg[q]]
          send topologie[q](p, FALSE, top)
       /* recepţionează topologiile de la vecini */
       for [q = 1 to N st leg[q]] {
          receive topologie[p](transm, qgata,
top_nou);
          top = top OR top_nou;
          if (qgata) activ[transm] = FALSE;
       }
       if (toate liniile din top au un element TRUE)
          gata = TRUE;
 }

 /* transmite top vecinilor activi
    în ultima rundă */
 for [q = 1 to N st activ[q]]
     send topologie[q](p,TRUE,top)
```

se executa D+1 runde

numar mesaje nu depaseste:

Numărul de mesaje =

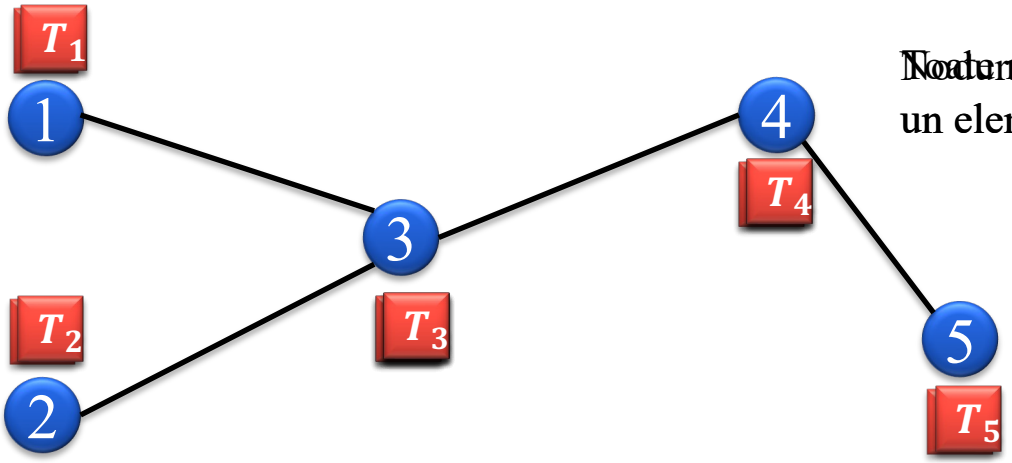
$2m(D+1)N$

m = numărul legăturilor

N = numărul de noduri

D = diametrul reţelei

Algoritmul pulsaţiilor (5)



Nodurile 3 și 4 au cel puțin un element ***true*** pe fiecare linie

Sau logic între tabela de adiacență proprie și cele primite de la vecini

$topologie[1..N]$

1	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

2	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

3	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

4	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

5	1	2	3	4	5
1			T		
2			T		
3	T	T		T	
4			T		T
5				T	

1	1	2	3	4	5
			F		

2	1	2	3	4	5
			F		

3	1	2	3	4	5
	T	T		F	

4	1	2	3	4	5
			F		T

5	1	2	3	4	5
				F	

$activ[1..N]$

Algoritmi cu mesaje de sondaj cu ecou

Algoritmi cu mesaje de sondaj cu ecou

- Ideea :
  - un **Iniţiator** colectează informaţii despre topologia locală a tuturor celorlalte noduri, determină topologia întregii reţele şi o difuzează nodurilor.
- **Cazul topologiilor arbore ⇒ sursa (initiatorul) este rădacina arborelui**
  - Două faze :
    - mesajele de sondaj se propagă de la inițiator spre frunze
    - mesajele de ecou *(care conțin topologii parțiale)* se propagă de la frunze spre inițiator.

Algoritmi cu mesaje de sondaj cu ecou (2)

```
const int inițiator = indexul nodului
inițiator;

typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondaj[1:N](int transm);
chan ecou[1:M]  (tip_top top);
chan ecou_final (tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = N + 1;
    tip_top top_nou;

    top[p, 1:n] = leg;
```

Algoritmi cu mesaje de sondaj cu ecou (3)

```
if (p <> inițiator)
   receive sondaj[p](părinte);

/* transmite sondaje celorlalte noduri vecine, copiii
lui p */
for [q = 1 to N st (leg[q] AND (q <> părinte))]
   send sondaj[q](p);


/* primește ecourile și fă reuniunea lor */
for [q = 1 to N st (leg[q] AND (q <> părinte))] {
   receive ecou[p](top_nou);
   top = top OR top_nou;
}


if (p <> inițiator)
   send ecou[părinte](top);
}
```

Algoritmi cu mesaje de sondaj cu ecou (4)

- **Cazul general**
  - după ce primeşte un prim mesaj de sondaj, un nod îl propagă tuturor celorlalţi vecini
  - nodul aşteaptă ecouri de la vecini
  - **există cicluri** $\Longrightarrow$ nodul poate primi și sondaje
    - topologia locală va fi transmisă doar ca ecou la primul sondaj
    - pentru restul sondajelor se transmite o topologie nulă
    - sondajele și ecourile trebuie recepţionate pe acelaşi canal

Algoritmi cu mesaje de sondaj cu ecou (5)

```
/* index inițiator */
const inițiator = indexul nodului inițiator;

enum fel{sondă, ecou};
typedef tip_leg bool[1:N];
typedef tip_top bool[1:N, 1:N];

chan sondă-ecou[1:N](fel tip_mesaj, int transm, tip_top
top);
chan ecou_final(tip_top top);

process Nod[p=1 to N]{
    tip_leg leg = vecinii_lui_p;
    tip_top top = ([N * N] FALSE);
    int părinte = n + 1;
    tip_top top_nou;
```

Algoritmi cu mesaje de sondaj cu ecou (6)

```
   /* actualizează vecinii lui p */
   top[p, 1:N] = leg;

   fel k; int transm;
   int nr_ecouri = 0;

   if (p <> inițiator)
      receive sondă-ecou[p](k, părinte, top_nou);

   /* transmite sondaje celorlalte noduri vecine, copiii lui p
*/
   for [q = 1 to N st (leg[q] AND (q <> părinte))] {
      send sondă-ecou[q](sondă, p, O);  /* topologie nulă */
      nr_ecouri = nr_ecouri + 1;
   }
```

Algoritmi cu mesaje de sondaj cu ecou (7)

```
   /* primește ecourile și fă reuniunea lor, sau sonde și
ignoră */
   while (nr_ecouri > 0) {
      receive sondă-ecou[p](k, transm, top_nou);

      if (k == sondă)
         send sondă-ecou[transm](ecou, p, 0);
      else if (k == ecou) {
         top = top OR top_nou;
         nr_ecouri = nr_ecouri - 1;
      }
   }

   if (p <> inițiator)
      send sondă-ecou[părinte](ecou, p, top);
```

Complexitate

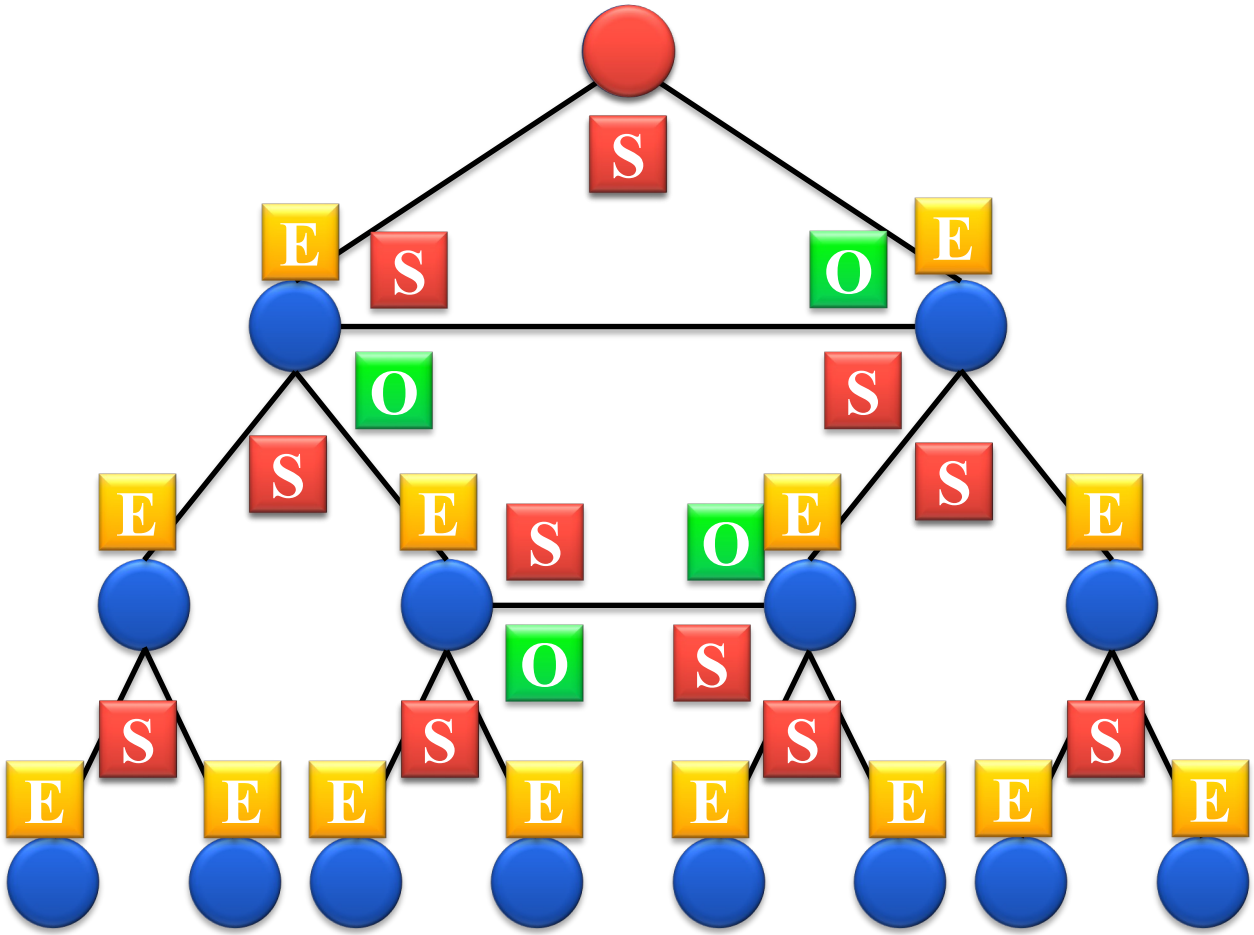
Numar mesaje

Fiecare legătură (aparţinând arborelui de acoperire) corespunzătoare primei sonde poartă două mesaje (sonda şi ecoul)

Celelalte, câte patru (două sonde şi două ecouri)

Transmiterea topologiei de la iniţiator la noduri necesită alte N mesaje.

Algoritmi cu mesaje de sondaj cu ecou (8)



Inițiator

Sondă

Ecou vid

Ecou

Sumar

- Difuzarea mesajelor prin inundare
- Difuzarea mesajelor folosind sondaje
- Problematica stabilirii topologiei
- Algoritmul pulsațiilor
- Algoritmi cu mesaje de sondaj cu ecou